

# Algorithm Development and Control Statements: Part 1

Chapter 5 of Visual C# How to Program, 6/e

# OBJECTIVES

In this chapter you'll:

- Learn basic problem-solving techniques.
- Develop algorithms through the process of top-down, stepwise refinement with pseudocode.
- Use the `if` and `if...else` selection statements to choose between actions.
- Use the `while` statement to execute statements repeatedly.
- Use counter-controlled iteration and sentinel-controlled iteration.
- Use nested control statements.
- Use the increment, decrement and compound assignment operators.
- Learn about the simple types.
- Understand the elements of structured programming with the sequence, selection and iteration control structures.

## 5.1 Introduction

## 5.2 Algorithms

## 5.3 Pseudocode

## 5.4 Control Structures

5.4.1 Sequence Structure

5.4.2 Selection Statements

5.4.3 Iteration Statements

5.4.4 Summary of Control Statements

## 5.5 **if** Single-Selection Statement

## 5.6 **if...else** Double-Selection Statement

5.6.1 Nested **if...else** Statements

5.6.2 Dangling-**else** Problem

5.6.3 Blocks

5.6.4 Conditional Operator (?:)

## 5.7 **Student** Class: Nested **if...else** Statements

## 5.8 **while** Iteration Statement

## 5.9 Formulating Algorithms: Counter-Controlled Iteration

5.9.1 Pseudocode Algorithm with Counter-Controlled Iteration

5.9.2 Implementing Counter-Controlled Iteration

5.9.3 Integer Division and Truncation

## 5.10 Formulating Algorithms: Sentinel-Controlled Iteration

5.10.1 Top-Down, Stepwise Refinement: The Top and First Refinement

5.10.2 Second Refinement

5.10.3 Implementing Sentinel-Controlled Iteration

5.10.4 Program Logic for Sentinel-Controlled Iteration

5.10.5 Braces in a **while** Statement

5.10.6 Converting Between Simple Types Explicitly and Implicitly

5.10.7 Formatting Floating-Point Numbers

## 5.11 Formulating Algorithms: Nested Control Statements

5.11.1 Problem Statement

5.11.2 Top-Down, Stepwise Refinement: Pseudocode Representation of the Top

5.11.3 Top-Down, Stepwise Refinement: First Refinement

5.11.4 Top-Down, Stepwise Refinement: Second Refinement

5.11.5 Complete Second Refinement of the Pseudocode

5.11.6 App That Implements the Pseudocode Algorithm

## 5.12 Compound Assignment Operators

## 5.13 Increment and Decrement Operators

5.13.1 Prefix Increment vs. Postfix Increment

5.13.2 Simplifying Increment Statements

5.13.3 Operator Precedence and Associativity

## 5.14 Simple Types

## 5.15 Wrap-Up

## 5.2 Algorithms

- ▶ Computers solve problems by executing a series of actions in a specific order.
- ▶ An algorithm is a procedure for solving a problem, in terms of:
  - the actions to be executed and
  - the order in which these actions are executed

## 5.2 Algorithms (Cont..)

- ▶ Consider the “rise-and-shine algorithm” followed by one junior executive for getting out of bed and going to work:
  - (1) get out of bed,
  - (2) take off pajamas,
  - (3) take a shower,
  - (4) get dressed,
  - (5) eat breakfast and
  - (6) carpool to work.

## 5.2 Algorithms (Cont.)

- ▶ Suppose that the same steps are performed in a different order:
  - (1) get out of bed,
  - (2) take off pajamas,
  - (3) get dressed,
  - (4) take a shower,
  - (5) eat breakfast,
  - (6) carpool to work.
- ▶ In this case, the junior executive shows up for work soaking wet.
- ▶ The order in which statements execute is called program control.



## 5.3 Pseudocode

- ▶ Pseudocode is similar to English—it's not a programming language.
- ▶ It helps you “think out” an app before attempting to write it.
- ▶ A carefully prepared pseudocode app can easily be converted to a corresponding C# app.

## 5.3 Pseudocode

- ▶ The pseudocode of Fig. 5.1 corresponds to the algorithm that inputs two integers from the user, adds these integers and displays their sum.
- ▶ The pseudocode statements are simply English statements that convey what task is to be performed in C#.

- 
- 1**    *Prompt the user to enter the first integer*
  - 2**    *Input the first integer*
  - 3**
  - 4**    *Prompt the user to enter the second integer*
  - 5**    *Input the second integer*
  - 6**
  - 7**    *Add first integer and second integer, store result*
  - 8**    *Display result*
- 

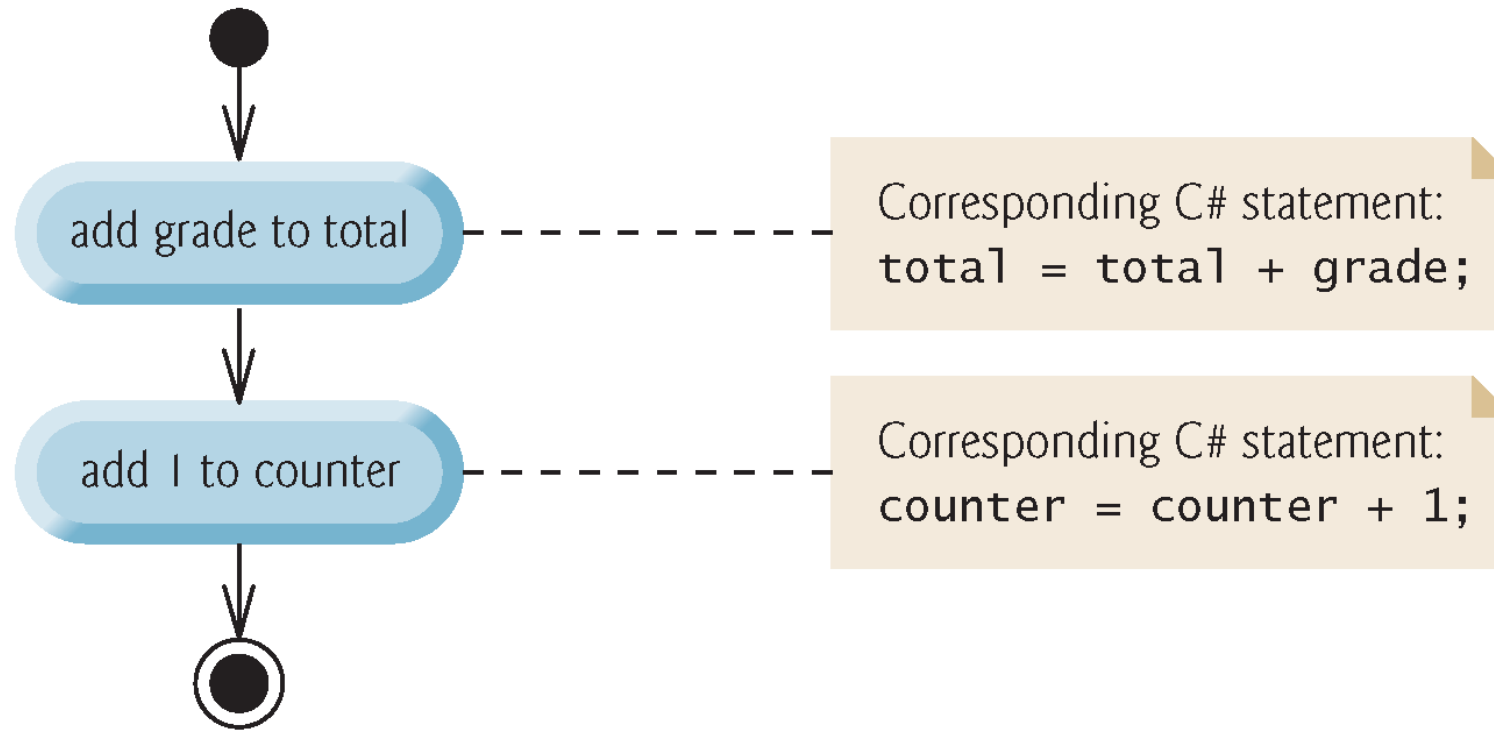
**Fig. 5.1** | Pseudocode for the addition program of Fig. 3.14.

## 5.4 Control Structures

- ▶ Normally, statements execute one after the other in sequential execution.
- ▶ Various C# statements enable you to specify the next statement to execute. This is called transfer of control.
- ▶ Structured apps are clearer, easier to debug and modify, and more likely to be bug free.

## 5.4.1 Sequence Structure

- ▶ The *sequence structure* is built into C#.
  - Unless directed otherwise, the computer executes C# statements one after the other in the order in which they're written—that is, in sequence.
- ▶ An activity diagram models the workflow of a software system (Fig. 5.1).
- ▶ Activity diagrams are composed of symbols such as action-state symbols, diamonds, small circles and notes.
- ▶ Transition arrows represent the flow of the activity.
- ▶ The solid circle represents the activity's initial state.
- ▶ The solid circle surrounded by a hollow circle represents the final state.



---

**Fig. 5.2** | Sequence structure activity diagram.

## 5.4.1 Sequence Structure

- ▶ Single-entry/single-exit control statements make it easy to build apps.
- ▶ Control statements are “attached” to one another by connecting the exit point of one to the entry point of the next.
- ▶ This procedure is called control-statement stacking.
- ▶ Control-statement nesting allows a control statement to appear inside another control statement.

## 5.4.2 Selection Statements

- ▶ C# has three types of selection structures, which from this point forward we shall refer to as **selection statements**.
  - The **if statement** performs (selects) an action if a condition is *true* or skips the action if the condition is *false*.
  - The **if...else statement** performs an action if a condition is *true* or performs a different action if the condition is *false*.
  - The **switch statement** (Chapter 6) performs one of many different actions, depending on the value of an expression.



## 5.4.3 Iteration Statements

- ▶ C# provides four **iteration statements** (sometimes called **repetition statements** or **looping statements**) that enable programs to perform statements repeatedly as long as a condition (called the **loop-continuation condition**) remains *true*.
  - `while`, `do...while`, `for` and `foreach` statements.
- ▶ The `while`, `for` and `foreach` statements perform the action (or group of actions) in their bodies *zero or more times*.
- ▶ The `do...while` statement performs the action (or group of actions) in its body *one or more times*.

## 5.4.4 Summary of Control Statements

- ▶ Every app is formed by combining as many of these statements as is appropriate for the algorithm the app implements.
- ▶ Each activity diagram contains an initial state and a final state that represent a control statement's entry point and exit point, respectively.
- ▶ **Single-entry/single-exit control statements** make it easy to build programs—we simply connect the exit point of one to the entry point of the next.
- ▶ We call this **control-statement stacking**.
- ▶ There's only one other way in which control statements- may be connected—**control-statement nesting**—in which one control statement appears *inside* another.

## 5.5 **if** Single-Selection Statement

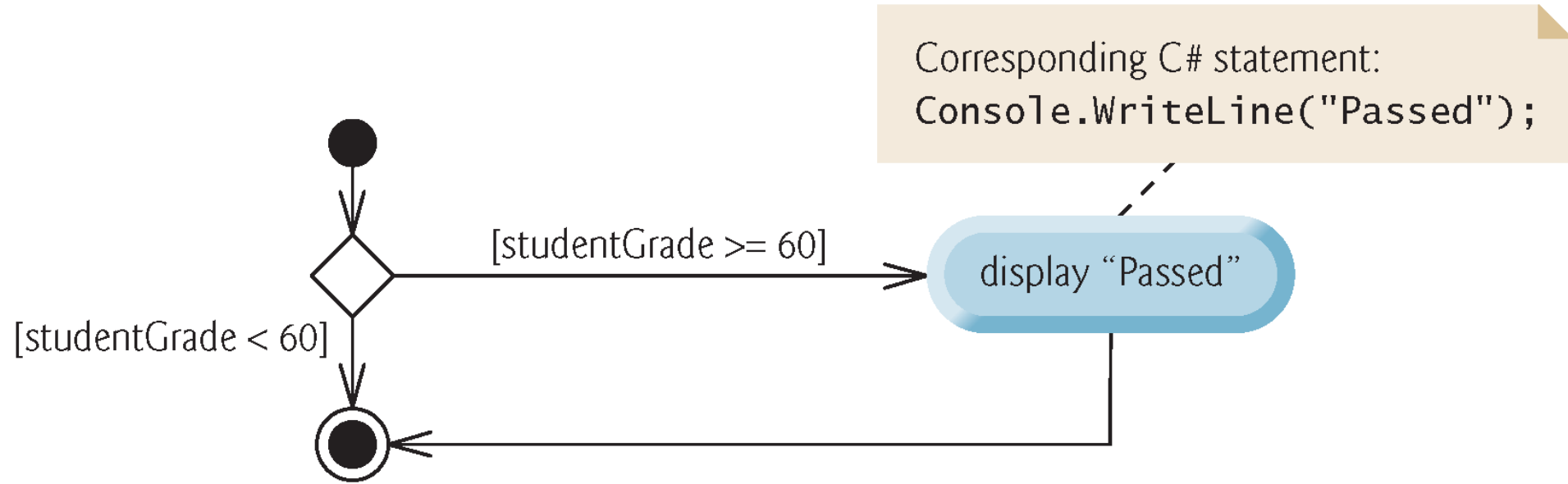
- ▶ *if grade is greater than or equal to 60  
display "Passed"*
- ▶ The preceding pseudocode if statement may be written in C# as:
  - `if (grade >= 60)`  
    {  
        `Console.WriteLine("Passed");`  
    }
- ▶ The C# code corresponds closely to the pseudocode.

## 5.5 `if` Single-Selection Statement (Cont..)

- ▶ ***bool Simple Type***
- ▶ You saw in Chapter 3 that decisions can be based on conditions containing relational or equality operators.
- ▶ A decision can be based on *any* expression that evaluates to **true** or **false**.
- ▶ C# provides the simple type **bool** for Boolean variables that can hold only the values **true** and **false**—each of these is a C# keyword.

## 5.5 **if** Single-Selection Statement (Cont..)

- ▶ Figure 5.2 illustrates the single-selection if statement.
- ▶ The diamond, or decision symbol indicates that a decision is to be made.
- ▶ The workflow will continue along a path determined by one of the symbol's two associated guard conditions.



**Fig. 5.3** | if single-selection statement UML activity diagram.

## 5.6 `if...else` Double-Selection Statement

- ▶ The `if...else` double-selection statement allows you to specify an action to perform when the condition is true and a different action when the condition is false

## 5.6 `if...else` Double-Selection Statement

- ▶ *if grade is greater than or equal to 60*  
    *display "Passed"*  
*else*  
    *display "Failed"*
- ▶ The preceding `if...else` pseudocode statement can be written in C# as
  - ```
if (grade >= 60)
{
    Console.WriteLine("Passed");
}
else
{
    Console.WriteLine("Failed");
}
```





## Good Programming Practice 5.1

*Indent both body statements of an `if...else` statement.  
Visual Studio does this for you.*

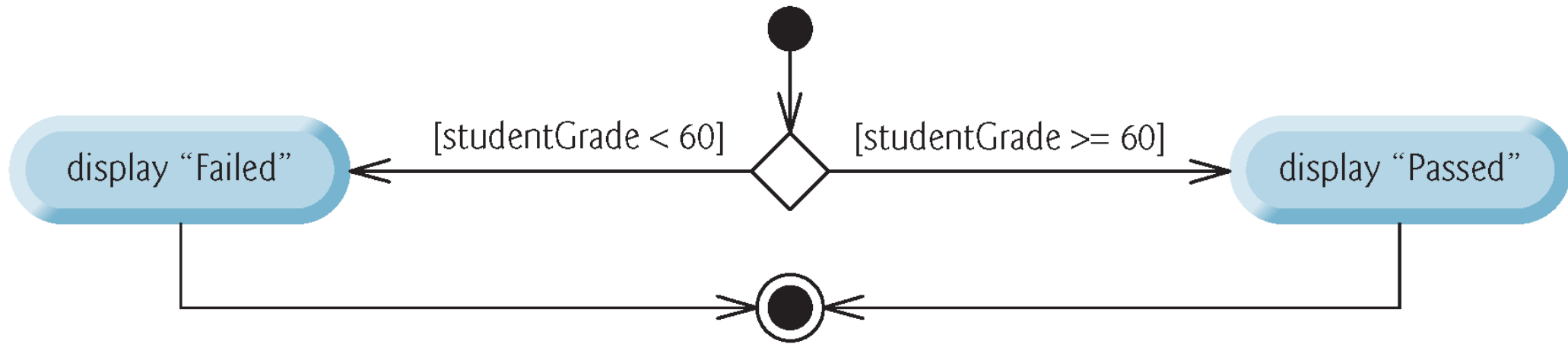


## Good Programming Practice 5.2

*If there are several levels of indentation, each level should be indented the same additional amount of space. Again, we use three-space indents for book-publication purposes, but Microsoft recommends four-space indents—the default in Visual Studio.*

## 5.6 **if...else** Double-Selection Statement (Cont.)

- ▶ Figure 5.4 illustrates the flow of control in the if...else statement.
- ▶ The symbols in the UML activity diagram represent action states and a decision.



---

**Fig. 5.4** | if...else double-selection statement UML activity diagram.

## 5.6.1 Nested `if...else` Statements

- ▶ Nested `if...else` Statements
- ▶ An app can test multiple cases with nested `if...else` statements:
- ▶ *if grade is greater than or equal to 90*

*display "A"*

*else*

*if grade is greater than or equal to 80*

*display "B"*

*else*

*if grade is greater than or equal to 70*

*display "C"*

*else*

*if grade is greater than or equal to 60*

*display "D"*

*else*

*display "F"*

## 5.6.1 Nested if...else Statements

```
▶ if (studentGrade >= 90)
{
    Console.WriteLine("A")
}
else
{
    if (studentGrade >= 80)
    {
        Console.WriteLine("B")
    }
    else
    {
        if (studentGrade >= 70)
        {
            Console.WriteLine("C")
        }
        else
        {
            if (studentGrade >= 60)
            {
                Console.WriteLine("D")
            }
            else
            {
                Console.WriteLine("F")
            }
        }
    }
}
```

## 5.6.1 Nested `if...else` Statements

```
▶ if (studentGrade >= 90)
{
    Console.WriteLine("A");
}
else if (studentGrade >= 80)
{
    Console.WriteLine("B");
}
else if (studentGrade >= 70)
{
    Console.WriteLine("C");
}
else if (studentGrade >= 60)
{
    Console.WriteLine("D");
}
else
{
    Console.WriteLine("F");
}
```



## Error-Prevention Tip 5.1

*In a nested `if...else` statement, ensure that you test for all possible cases.*



## 5.6.2 Dangling-else Problem

- ▶ Throughout the text, we always enclose control statement bodies in braces ( { and } ). This avoids a logic error called the “dangling-else” problem.
- ▶ We investigate this problem in Exercises 5.27-5.29.

## 5.6.3 Blocks

- ▶ Statements contained within a pair of braces ({ and }) form a **block**:
  - ```
if (studentGrade >= 60)
{
    Console.WriteLine("Passed");
}
else
{
    Console.WriteLine("Failed");
    Console.WriteLine("You must take this course again.");
}
```
- ▶ Without the braces, the last statement in the `else` block would execute regardless of whether the grade was less than 60.

## 5.6.3 Blocks

- ▶ A logic error has its effect at execution time.
  - A fatal logic error causes an app to fail and terminate prematurely.
  - A nonfatal logic error allows an app to continue executing.



## Common Programming Error 5.1

*Placing a semicolon after the parenthesized condition in an `if` or `if...else` statement leads to a logic error in single-selection `if` statements and a syntax error in double-selection `if...else` statements (when the `if`-part contains a body statement).*

## 5.6.4 Conditional Operator (?:)

- ▶ The conditional operator (?:) can be used in place of an `if...else` statement.
- ▶ `Console.WriteLine(  
    studentGrade >= 60 ? "Passed" :  
    "Failed");`
  - The first operand is a `boolean` expression that evaluates to `true` or `false`.
  - The second operand is the value if the expression is `true`
  - The third operand is the value if the expression is `false`.

## 5.7 Student Class: Nested `if...else` Statements

- ▶ The example of Figs. 5.5–5.6 demonstrates a nested `if...else` statement that determines a student's letter grade based on the student's average in a course.
- ▶ Read-only property `LetterGrade` (lines 39–68) uses *nested if...else statements* to determine the Student's letter grade based on the Student's average.
- ▶ A read-only property provides only a `get accessor`.
- ▶ Local variable `letterGrade` is initialized to `string.Empty` (line 43), which represents the empty `string` (that is, a `string` containing no characters).

---

```
1  // Fig. 5.5: Student.cs
2  // Student class that stores a student name and average.
3  using System;
4
5  class Student
6  {
7      public string Name { get; set; } // property
8      private int average; // instance variable
9
10     // constructor initializes Name and Average properties
11     public Student(string studentName, int studentAverage)
12     {
13         Name = studentName;
14         Average = studentAverage; // sets average instance variable
15     }
16
```

---

**Fig. 5.5** | Student class that stores a student name and average. (Part 1 of 4.)

---

```
17 // property to get and set instance variable average
18 public int Average
19 {
20     get // returns the Student's average
21     {
22         return average;
23     }
24     set // sets the Student's average
25     {
26         // validate that value is > 0 and <= 100; otherwise,
27         // keep instance variable average's current value
28         if (value > 0)
29         {
30             if (value <= 100)
31             {
32                 average = value; // assign to instance variable
33             }
34         }
35     }
36 }
37
```

---

**Fig. 5.5** | Student class that stores a student name and average. (Part 2 of 4.)



---

```
38 // returns the Student's letter grade, based on the average
39 string LetterGrade
40 {
41     get
42     {
43         string letterGrade = string.Empty; // string.Empty is ""
44
45         if (average >= 90)
46         {
47             letterGrade = "A";
48         }
49         else if (average >= 80)
50         {
51             letterGrade = "B";
52         }
```

---

**Fig. 5.5** | Student class that stores a student name and average. (Part 3 of 4.)

---

```
53         else if (average >= 70)
54         {
55             letterGrade = "C";
56         }
57         else if (average >= 60)
58         {
59             letterGrade = "D";
60         }
61         else
62         {
63             letterGrade = "F";
64         }
65
66         return letterGrade;
67     }
68 }
69 }
```

---

**Fig. 5.5** | Student class that stores a student name and average. (Part 4 of 4.)

```
1  // Fig. 5.6: StudentTest.cs
2  // Create and test Student objects.
3  using System;
4
5  class StudentTest
6  {
7      static void Main()
8      {
9          Student student1 = new Student("Jane Green", 93);
10         Student student2 = new Student("John Blue", 72);
11
12         Console.Write($"{student1.Name}'s letter grade equivalent of ");
13         Console.WriteLine($"{student1.Average} is {student1.LetterGrade}");
14         Console.Write($"{student2.Name}'s letter grade equivalent of ");
15         Console.WriteLine($"{student2.Average} is {student2.LetterGrade}");
16     }
17 }
```

Jane Green's letter grade equivalent of 93 is A  
John Blue's letter grade equivalent of 72 is C

**Fig. 5.6** | Create and test Student objects.

## 5.8 while Iteration Statement

- ▶ A iteration statement allows you to specify that an app should repeat an action:
- ▶ *While there are more items on my shopping list  
put next item in cart and cross it off my list*
- ▶ As an example of C#'s `while` iteration statement, consider a code segment designed to find the first power of 3 larger than 100:
- ▶ `int product = 3;`

```
while (product <= 100)
{
    product = 3 * product;
}
```

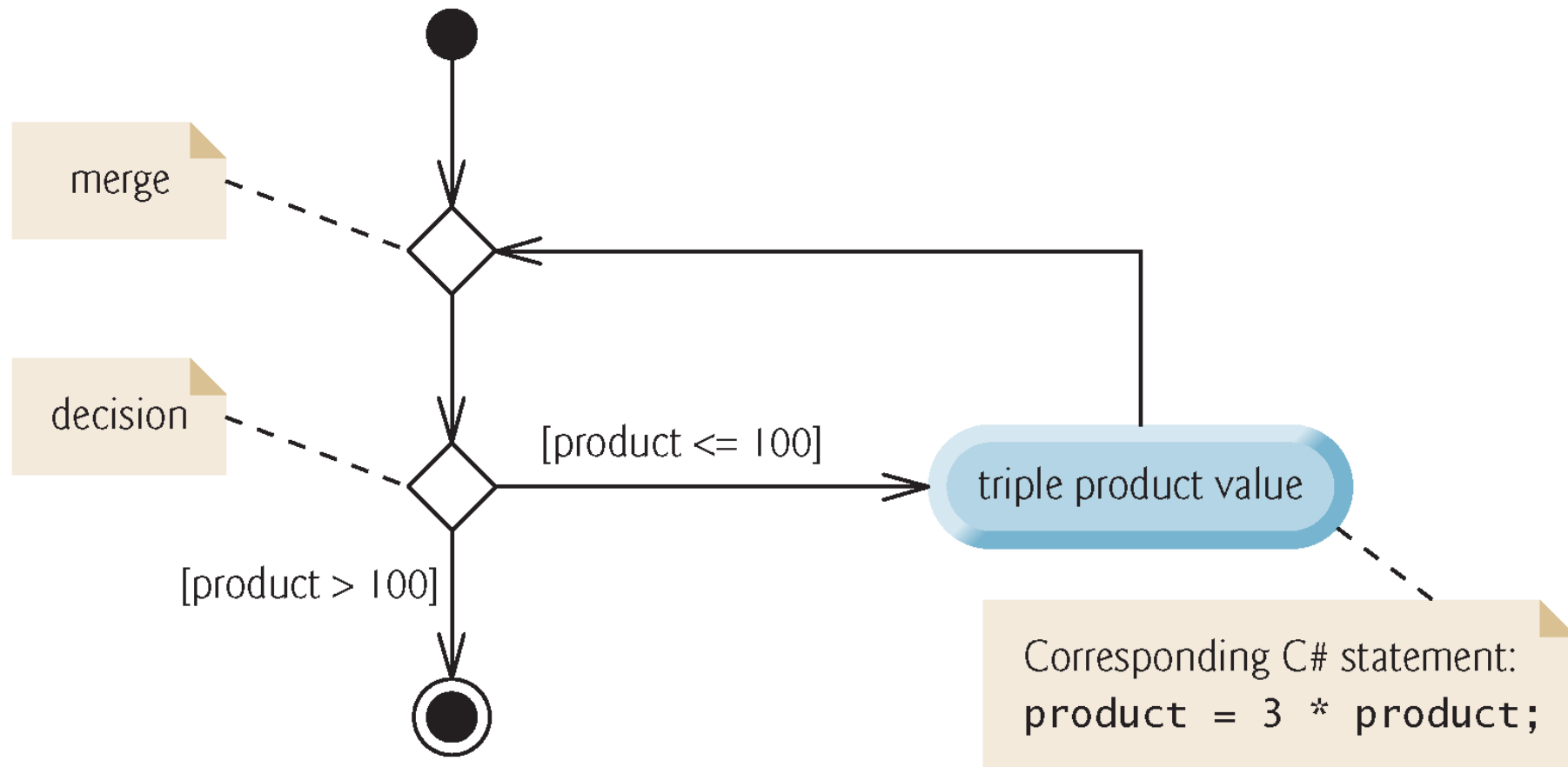


## Common Programming Error 5.2

*Not providing in the body of a `while` statement an action that eventually causes the condition in the `while` to become false results in a logic error called an **infinite loop** (the loop never terminates). This often will make your program appear to “hang.”*

## 5.8 `while` Iteration Statement

- ▶ The UML activity diagram in Fig. 5.7 illustrates the flow of control in the preceding `while` statement.
- ▶ This diagram introduces the UML's **merge symbol**.
- ▶ The UML represents *both* the merge symbol and the decision symbol as diamonds.
- ▶ The merge symbol joins multiple flows of activity into one.
  - In this diagram, the merge symbol joins the transitions from the initial state and from the action state, so they both flow into the decision that determines whether the loop should begin (or continue) executing



**Fig. 5.7** | while iteration statement UML activity diagram.

## 5.9 Formulating Algorithms: Counter-Controlled Iteration

- ▶ Consider the following problem statement:
- ▶ A class of 10 students took a quiz. The grades (integers in the range 0 to 100) for this quiz are available to you.
- ▶ Determine the class average on the quiz.
- ▶ The algorithm must input each grade, keep track of the total of all grades input, perform the averaging calculation and display the result.



## 5.9.1 Pseudocode Algorithm with Counter-Controlled Iteration

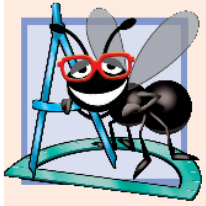
- ▶ Pseudocode Algorithm with Counter-Controlled Iteration
- ▶ We use counter-controlled iteration to input the grades one at a time.
- ▶ A variable called a counter controls the number of times a set of statements will execute.
- ▶ Counter-controlled iteration is often called definite iteration, because the number of iterations is known before the loop begins executing.
- ▶ Figure 5.8 expresses the algorithm in pseudocode.
- ▶ A **total** is a variable used to accumulate the sum of several values.
- ▶ A **counter** is a variable used to count.

---

```
1  Set total to zero
2  Set grade counter to one
3
4  While grade counter is less than or equal to 10
5      Prompt the user to enter the next grade
6      Input the next grade
7      Add the grade into the total
8      Add one to the grade counter
9
10 Set the class average to the total divided by 10
11 Display the class average
```

---

**Fig. 5.8** | Pseudocode algorithm that uses counter-controlled iteration to solve the class-average problem.



## Software Engineering Observation 5.1

*Experience has shown that the most difficult part of solving a problem on a computer is developing the algorithm for the solution. Once a correct algorithm has been specified, producing a working C# program from it is usually straightforward.*

## 5.9.2 Implementing Counter-Controlled Iteration

- ▶ In Fig. 5.9, method `Main` implements the class-averaging algorithm described by the pseudocode in —it allows the user to enter 10 grades, then calculates and displays the average

---

```
1  // Fig. 5.9: ClassAverage.cs
2  // Solving the class-average problem using counter-controlled iteration.
3  using System;
4
5  class ClassAverage
6  {
7      static void Main()
8      {
9          // initialization phase
10         int total = 0; // initialize sum of grades entered by the user
11         int gradeCounter = 1; // initialize grade # to be entered next
12
```

---

**Fig. 5.9** | Solving the class-average problem using counter-controlled iteration. (Part 1 of 3.)

```
13 // processing phase uses counter-controlled iteration
14 while (gradeCounter <= 10) // loop 10 times
15 {
16     Console.Write("Enter grade: "); // prompt
17     int grade = int.Parse(Console.ReadLine()); // input grade
18     total = total + grade; // add the grade to total
19     gradeCounter = gradeCounter + 1; // increment the counter by 1
20 }
21
22 // termination phase
23 int average = total / 10; // integer division yields integer result
24
25 // display total and average of grades
26 Console.WriteLine($"\\nTotal of all 10 grades is {total}");
27 Console.WriteLine($"Class average is {average}");
28 }
29 }
```

**Fig. 5.9** | Solving the class-average problem using counter-controlled iteration. (Part 2 of 3.)

```
Enter grade: 88
Enter grade: 79
Enter grade: 95
Enter grade: 100
Enter grade: 48
Enter grade: 88
Enter grade: 92
Enter grade: 83
Enter grade: 90
Enter grade: 85
```

```
Total of all 10 grades is 848
Class average is 84
```

**Fig. 5.9** | Solving the class-average problem using counter-controlled iteration. (Part 3 of 3.)

## 5.9.2 Implementing Counter-Controlled Iteration

- ▶ C# requires local variables to be **definitely assigned**—that is, each local variable must be assigned a value before the variable's value is used.





## Common Programming Error 5.3

*All local variables must be definitely assigned before their values are used in expressions. Using a local variable's value before it's definitely assigned results in a compilation error.*



## Error-Prevention Tip 5.2

*Initializing local variables when they're declared helps you avoid compilation errors that might arise from attempts to use uninitialized data. While C# does not require that local-variable initializations be incorporated into declarations, it does require that local variables be initialized before their values are used in an expression.*



### **Error-Prevention Tip 5.3**

*Initialize each counter and total, either in its declaration or in an assignment statement. Totals are normally initialized to 0. Counters are normally initialized to 0 or 1, depending on how they're used (we'll show examples of each).*

## 5.9.3 Integer Division and Truncation

- ▶ Dividing two integers results in integer division.
- ▶ Any fractional part of the result is lost (i.e., truncated, not rounded).



## Common Programming Error 5.4

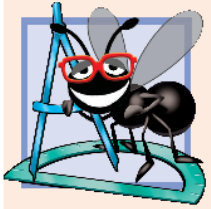
*Assuming that integer division rounds (rather than truncates) can lead to incorrect results. For example,  $7 \div 4$ , which yields 1.75 in conventional arithmetic, truncates to 1 in integer arithmetic, rather than rounding to 2.*

## 5.10 Formulating Algorithms: Sentinel-Controlled Iteration

- ▶ Consider the following problem:
  - Develop a class-averaging app that processes grades for an arbitrary number of students each time it's run.
- ▶ In this example, no indication is given of how many grades the user will enter during the app's execution.
- ▶ A sentinel value can be entered to indicate “end of data entry.” This is called sentinel-controlled iteration.
- ▶ Sentinel-controlled iteration is often called indefinite iteration because the number of iterations is not known before the loop begins.

## 5.10.1 Top-Down, Stepwise Refinement: The Top and First Refinement

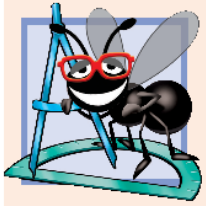
- ▶ We approach the class averaging program with a technique called top-down, stepwise refinement.
- ▶ The top conveys the overall function of the app: determine the class average for the quiz
- ▶ We divide the top into a series of smaller tasks in the first refinement:
  - *initialize variables*
  - *input, sum and count the quiz grades*
  - *calculate and display the class average*



## Software Engineering Observation 5.2

*Each refinement, as well as the top itself, is a complete specification of the algorithm—only the level of detail varies.*





## Software Engineering Observation 5.3

*Many programs can be divided logically into three phases: an initialization phase that initializes the variables; a processing phase that inputs data values and adjusts program variables accordingly; and a termination phase that calculates and outputs the final results.*

## 5.10.2 Second Refinement

- ▶ The second refinement specifies individual variables.
  - *initialize variables*
- ▶ This can be refined as follows:
  - *initialize total to zero*
  - *initialize counter to zero*

## 5.10.2 Second Refinement

- ▶ *input, sum and count the quiz grades*
- ▶ The second refinement of the preceding pseudocode statement is:
  - *prompt the user to enter the first grade*
  - *input the first grade (possibly the sentinel)*
  - *while the user has not yet entered the sentinel*
    - add this grade into the running total*
    - add one to the grade counter*
    - prompt the user to enter the next grade*
    - input the next grade (possibly the sentinel)*

## 5.10.2 Second Refinement

- ▶ *calculate and display the class average*
- ▶ The preceding pseudocode can be refined as follows:
  - *if the counter is not equal to zero*
    - set the average to the total divided by the counter*
    - display the average*
  - else*
    - display “No grades were entered”*
- ▶ We are careful here to test for the possibility of division by zero.
- ▶ The complete second refinement of the pseudocode for the class-average problem is shown in Fig. 5.10.



## Error-Prevention Tip 5.4

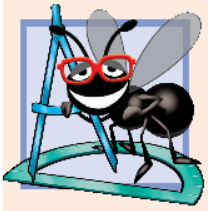
*When performing division by an expression whose value could be zero, explicitly test for this possibility and handle it appropriately in your app (e.g., by displaying an error message) rather than allowing the error to occur.*

---

```
1  Initialize total to zero
2  Initialize counter to zero
3
4  Prompt the user to enter the first grade
5  Input the first grade (possibly the sentinel)
6
7  While the user has not yet entered the sentinel
8      Add this grade into the running total
9      Add one to the grade counter
10     Prompt the user to enter the next grade
11     Input the next grade (possibly the sentinel)
12
13     If the counter is not equal to zero
14         Set the average to the total divided by the counter
15         Print the average
16     Else
17         Print "No grades were entered"
```

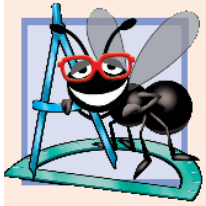
---

**Fig. 5.10** | Class-averaging pseudocode algorithm with sentinel-controlled iteration.



## Software Engineering Observation 5.4

*Terminate the top-down, stepwise refinement process when you've specified the pseudocode algorithm in sufficient detail for you to convert the pseudocode to C#.*



## Software Engineering Observation 5.5

*Some programmers do not use program-development tools like pseudocode. They feel that their ultimate goal is to solve the problem on a computer and that writing pseudocode merely delays the production of final outputs. Although this may work for simple and familiar problems, it can lead to serious errors and delays in large, complex projects.*



## 5.10.3 Implementing Sentinel-Controlled Iteration

- ▶ In Fig. 5.11, method `Main` implements the pseudocode algorithm of Fig. 5.10.
- ▶ Although each grade entered by the user is an integer, the averaging calculation is likely to produce a number with a decimal.
- ▶ The type `int` cannot represent such a number.
- ▶ C# provides data types `float` and `double` to store floating-point numbers in memory.

---

```
1  // Fig. 5.11: ClassAverage.cs
2  // Solving the class-average problem using sentinel-controlled iteration.
3  using System;
4
5  class ClassAverage
6  {
7      static void Main()
8      {
9          // initialization phase
10         int total = 0; // initialize sum of grades
11         int gradeCounter = 0; // initialize # of grades entered so far
12
13         // processing phase
14         // prompt for input and read grade from user
15         Console.Write("Enter grade or -1 to quit: ");
16         int grade = int.Parse(Console.ReadLine());
17
```

---

**Fig. 5.11** | Solving the class-average problem using sentinel-controlled iteration. (Part 1 of 4.)

---

```
18 // loop until sentinel value is read from the user
19 while (grade != -1)
20 {
21     total = total + grade; // add grade to total
22     gradeCounter = gradeCounter + 1; // increment counter
23
24     // prompt for input and read grade from user
25     Console.Write("Enter grade or -1 to quit: ");
26     grade = int.Parse(Console.ReadLine());
27 }
28
```

---

**Fig. 5.11** | Solving the class-average problem using sentinel-controlled iteration. (Part 2 of 4.)

---

```
29      // termination phase
30      // if the user entered at least one grade...
31      if (gradeCounter != 0)
32      {
33          // use number with decimal point to calculate average of grades
34          double average = (double) total / gradeCounter;
35
36          // display the total and average (with two digits of precision)
37          Console.WriteLine(
38              $"\\nTotal of the {gradeCounter} grades entered is {total}");
39          Console.WriteLine($"Class average is {average:F}");
40      }
41      else // no grades were entered, so output error message
42      {
43          Console.WriteLine("No grades were entered");
44      }
45  }
46 }
```

---

**Fig. 5.11** | Solving the class-average problem using sentinel-controlled iteration. (Part 3 of 4.)

```
Enter grade or -1 to quit: 97
Enter grade or -1 to quit: 88
Enter grade or -1 to quit: 72
Enter grade or -1 to quit: -1

Total of the 3 grades entered is 257
Class average is 85.67
```

**Fig. 5.11** | Solving the class-average problem using sentinel-controlled iteration. (Part 4 of 4.)



## Good Programming Practice 5.3

*In a sentinel-controlled loop, prompts should remind the user of the sentinel.*



## Error-Prevention Tip 5.5

*Omitting the braces that delimit a block can lead to logic errors, such as infinite loops. To prevent this and other problems, we always enclose the body of every control statement in braces even if the body contains only a single statement.*

## 5.10.6 Converting Between Simple Types Explicitly and Implicitly

- ▶ To perform a floating-point calculation with integer values, we temporarily treat these values as floating-point numbers.
- ▶ A unary cast operator such as `(double)` performs explicit conversion.
- ▶ C# performs an operation called promotion (or implicit conversion) on selected operands for use in the expression.
- ▶ The cast operator is formed by placing parentheses around the name of a type.
- ▶ This operator is a unary operator (i.e., an operator that takes only one operand).
- ▶ Cast operators have the second highest precedence.



## 5.10.7 Formatting Floating-Point Numbers

- ▶ Line 39
  - `Console.WriteLine($"Class average is {average:F}");`
- ▶ outputs the class average.
- ▶ The **format specifier F** in the interpolation expression
  - `{average:F}`
- ▶ typically formats average's value with two digits to the right of the decimal point
  - the Windows culture settings on the user's machine determine the actual format, including the digits to the right of the decimal point, whether commas or periods are used for separating thousands, millions, etc.

## 5.11 Formulating Algorithms: Nested Control Statements

- ▶ We've seen that control statements can be *stacked* on top of one another (in sequence).
- ▶ In this case study, we examine the only other structured way control statements can be connected, namely, by **nesting** one control statement within another.

## 5.11.1 Formulating Algorithms: Nested Control Statements

- ▶ A college offers a course that prepares students for the state licensing exam for real estate brokers. Last year, 10 of the students who completed this course took the exam. The college wants to know how well its students did on the exam. You've been asked to write an app to summarize the results. You've been given a list of these 10 students. Next to each name is written a 1 if the student passed the exam or a 2 if the student failed.

## 5.11.1 Formulating Algorithms: Nested Control Statements

- ▶ Your app should analyze the results of the exam as follows:
- ▶ Input each test result (i.e., a 1 or a 2). Display the message “Enter result” on the screen each time the app requests another test result.
- ▶ Count the number of test results of each type.
- ▶ Display a summary of the test results indicating the number of students who passed and the number who failed.
- ▶ If more than eight students passed the exam, display the message “Bonus to instructor!”

## 5.11.1 Formulating Algorithms: Nested Control Statements

- ▶ After reading the problem statement, we make the following observations:
  - The app must process test results for 10 students. A counter-controlled loop can be used because the number of test results is known in advance.
  - Each test result has a numeric value—either a 1 or a 2. Each time the app reads a test result, the app must determine whether the number is a 1 or a 2. We test for a 1 in our algorithm. If the number is not a 1, we assume that it's a 2.
  - Two counters are used to keep track of the exam results—one to count the number of students who passed the exam and one to count the number of students who failed the exam.
  - After the app has processed all the results, it must determine whether more than eight students passed the exam.

## 5.11.2 Top-Down, Stepwise Refinement: Pseudocode Representation of the Top

- ▶ Proceed with top-down, stepwise refinement:
  - *analyze exam results and decide whether the instructor should receive a bonus*

## 5.11.3 Top-Down, Stepwise Refinement: First Refinement

- ▶ Proceed with top-down, stepwise refinement:
  - *analyze exam results and decide whether the instructor should receive a bonus*
- ▶ Our first refinement is
  - *initialize variables*
  - *input the 10 exam results, and count passes and failures*
  - *display a summary of the exam results and decide whether the instructor should receive a bonus*

## 5.11.4 Top-Down, Stepwise Refinement: Second Refinement

- ▶ *initialize variables*
- ▶ The preceding pseudocode statement can be refined as follows:
  - *initialize passes to zero*
  - *initialize failures to zero*
  - *initialize student counter to one*



## 5.11.4 Top-Down, Stepwise Refinement: Second Refinement

- ▶ *input the 10 exam results, and count passes and failures*
- ▶ A nested double-selection statement will determine whether each exam result is a pass or a failure:
  - *while student counter is less than or equal to 10*
    - prompt the user to enter the next exam result*
    - input the next exam result*
    - 
    - if the student passed*
      - add one to passes*
    - else*
      - add one to failures*
    - 
    - add one to student counter*

## 5.11.4 Top-Down, Stepwise Refinement: Second Refinement

- ▶ *display a summary of the exam results and decide whether the instructor should receive a bonus*
- ▶ can be refined as follows:
  - *display the number of passes*
  - *display the number of failures*
  - *if more than eight students passed  
display “Bonus to Instructor!”*

## 5.11.5 Complete Second Refinement of the Pseudocode

- ▶ The complete second refinement of the pseudocode appears in Fig. 5.12.

---

```
1  Initialize passes to zero
2  Initialize failures to zero
3  Initialize student counter to one
4
5  While student counter is less than or equal to 10
6      Prompt the user to enter the next exam result
7      Input the next exam result
8
9      If the student passed
10         Add one to passes
11     Else
12         Add one to failures
13
14     Add one to student counter
15
16 Display the number of passes
17 Display the number of failures
18
19 If more than eight students passed
20     Display "Bonus to instructor!"
```

---

**Fig. 5.12** | Pseudocode for the examination-results problem.

## 5.11.6 App That Implements the Pseudocode Algorithm

- ▶ The program that implements the algorithm is shown in Fig. 5.13.

---

```
1  // Fig. 5.13: Analysis.cs
2  // Analysis of examination results, using nested control statements.
3  using System;
4
5  class Analysis
6  {
7      static void Main()
8      {
9          // initialize variables in declarations
10         int passes = 0; // number of passes
11         int failures = 0; // number of failures
12         int studentCounter = 1; // student counter
13
```

---

**Fig. 5.13** | Analysis of examination results, using nested control statements. (Part 1 of 5.)

---

```
14 // process 10 students using counter-controlled iteration
15 while (studentCounter <= 10)
16 {
17     // prompt user for input and obtain a value from the user
18     Console.WriteLine("Enter result (1 = pass, 2 = fail): ");
19     int result = int.Parse(Console.ReadLine());
20
21     // if...else is nested in the while statement
22     if (result == 1)
23     {
24         passes = passes + 1; // increment passes
25     }
26     else
27     {
28         failures = failures + 1; // increment failures
29     }
30
31     // increment studentCounter so loop eventually terminates
32     studentCounter = studentCounter + 1;
33 }
```

---

**Fig. 5.13** | Analysis of examination results, using nested control statements. (Part 2 of 5.)

---

```
34
35     // termination phase; prepare and display results
36     Console.WriteLine($"Passed: {passes}\nFailed: {failures}");
37
38     // determine whether more than 8 students passed
39     if (passes > 8)
40     {
41         Console.WriteLine("Bonus to instructor!");
42     }
43 }
44 }
```

---

**Fig. 5.13** | Analysis of examination results, using nested control statements. (Part 3 of 5.)



```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed: 9
Failed: 1
Bonus to instructor!
```

**Fig. 5.13** | Analysis of examination results, using nested control statements. (Part 4 of 5.)

```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Passed: 5
Failed: 5
```

**Fig. 5.13** | Analysis of examination results, using nested control statements. (Part 5 of 5.)

## 5.12 Compound Assignment Operators

- C# provides several **compound assignment operators**
- For example, you can abbreviate the statement

`C = C + 3;`

- with the **addition compound assignment operator**, **+=**, as

`C += 3;`

## 5.12 Compound Assignment Operators (Cont..)

- ▶ Figure 5.14 shows the arithmetic compound assignment operators, sample expressions using the operators and explanations of what the operators do.

| Assignment operator                                                 | Sample expression   | Explanation            | Assigns |
|---------------------------------------------------------------------|---------------------|------------------------|---------|
| <i>Assume:</i> <code>int c = 3, d = 5, e = 4, f = 6, g = 12;</code> |                     |                        |         |
| <code>+=</code>                                                     | <code>c += 7</code> | <code>c = c + 7</code> | 10 to c |
| <code>-=</code>                                                     | <code>d -= 4</code> | <code>d = d - 4</code> | 1 to d  |
| <code>*=</code>                                                     | <code>e *= 5</code> | <code>e = e * 5</code> | 20 to e |
| <code>/=</code>                                                     | <code>f /= 3</code> | <code>f = f / 3</code> | 2 to f  |
| <code>%=</code>                                                     | <code>g %= 9</code> | <code>g = g % 9</code> | 3 to g  |

**Fig. 5.14** | Arithmetic compound assignment operators.

## 5.13 Increment and Decrement Operators

- ▶ C# provides operators for adding or subtracting 1 from a numeric variable (Fig. 5.15).
  - The unary increment operator, ++
  - The unary decrement operator, --.

| Operator                            | Sample expression | Explanation                                                                                                                      |
|-------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>++</code> (prefix increment)  | <code>++a</code>  | Increments <code>a</code> by 1 and uses the new value of <code>a</code> in the expression in which <code>a</code> resides.       |
| <code>++</code> (postfix increment) | <code>a++</code>  | Increments <code>a</code> by 1, but uses the original value of <code>a</code> in the expression in which <code>a</code> resides. |
| <code>--</code> (prefix decrement)  | <code>--b</code>  | Decrements <code>b</code> by 1 and uses the new value of <code>b</code> in the expression in which <code>b</code> resides.       |
| <code>--</code> (postfix decrement) | <code>b--</code>  | Decrements <code>b</code> by 1 but uses the original value of <code>b</code> in the expression in which <code>b</code> resides.  |

**Fig. 5.15** | Increment and decrement operators.



## Good Programming Practice 5.4

*Unlike binary operators, the unary increment and decrement operators as a matter of style should be placed next to their operands, with no intervening spaces.*



## 5.13.1 Prefix Increment vs. Postfix Increment

- ▶ Figure 5.16 demonstrates the difference between the prefix increment and postfix increment versions of the ++ increment operator.

---

```
1  // Fig. 5.16: Increment.cs
2  // Prefix increment and postfix increment operators.
3  using System;
4
5  class Increment
6  {
7      static void Main()
8      {
9          // demonstrate postfix increment operator
10         int c = 5; // assign 5 to c
11         Console.WriteLine($"c before postincrement: {c}"); // displays 5
12         Console.WriteLine($"      postincrementing c: {c++}"); // displays 5
13         Console.WriteLine($"c after postincrement: {c}"); // displays 6
14
15         Console.WriteLine(); // skip a line
16
```

---

**Fig. 5.16** | Prefix-increment and postfix-increment operators. (Part 1 of 2.)

```
17      // demonstrate prefix increment operator
18      c = 5; // assign 5 to c
19      Console.WriteLine($" c before preincrement: {c}"); // displays 5
20      Console.WriteLine($"      preincrementing c: {++c}"); // displays 6
21      Console.WriteLine($" c after preincrement: {c}"); // displays 6
22  }
23 }
```

```
c before postincrement: 5
  postincrementing c: 5
c after postincrement: 6
```

```
c before preincrement: 5
  preincrementing c: 6
c after preincrement: 6
```

**Fig. 5.16** | Prefix-increment and postfix-increment operators. (Part 2 of 2.)

## 5.13.2 Simplifying Increment Statements

- ▶ `passes = passes + 1;`  
`failures = failures + 1;`  
`studentCounter = studentCounter + 1;`
- ▶ can be written more concisely with compound assignment operators:
- ▶ `passes += 1;`  
`failures += 1;`  
`studentCounter += 1;`

## 5.13.2 Simplifying Increment Statements

- ▶ This is even more precise with prefix increment operators:
  - ▶ `++passes;`  
`++failures;`  
`++studentCounter;`
- ▶ Or with postfix increment operators:
  - ▶ `passes++;`  
`failures++;`  
`studentCounter++;`



## Common Programming Error 5.5

*Attempting to use the increment or decrement operator on an expression other than one to which a value can be assigned is a syntax error. For example, writing  $++(x + 1)$  is a syntax error, because  $(x + 1)$  is not an expression to which a value can be assigned.*

## 5.13.3 Operator Precedence and Associativity

- ▶ Figure 5.17 shows the precedence and associativity of the operators we've introduced to this point.

| Operators |     |                      |                      |                 |    | Associativity | Type               |
|-----------|-----|----------------------|----------------------|-----------------|----|---------------|--------------------|
| .         | new | ++( <i>postfix</i> ) | --( <i>postfix</i> ) |                 |    | left to right | highest precedence |
| ++        | --  | +                    | -                    | ( <i>type</i> ) |    | right to left | unary prefix       |
| *         | /   | %                    |                      |                 |    | left to right | multiplicative     |
| +         | -   |                      |                      |                 |    | left to right | additive           |
| <         | <=  | >                    | >=                   |                 |    | left to right | relational         |
| ==        | !=  |                      |                      |                 |    | left to right | equality           |
| ?:        |     |                      |                      |                 |    | right to left | conditional        |
| =         | +=  | -=                   | *=                   | /=              | %= | right to left | assignment         |

**Fig. 5.17** | Precedence and associativity of the operators discussed so far.



## 5.14 Simple Types

- The table in Appendix B, Simple Types, lists the 13 **simple types** in C#.
- C# requires *all* variables to have a type.
- Instance variables of types `char`, `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `float`, `double`, and `decimal` are all given the value `0` by default.
- Instance variables of type `bool` are given the value `false` by default.
- ▶ Reference-type instance variables are initialized by default to the value `null`.